

Design of a Secure Mail Transfer System

With End-to-End Encryption, Bit-Level framing, CRC-32 Error Detection, Sliding Window Flow Control, and Heuristic Spam/Intrusion Monitoring

PROJECT TEAM MEMBERS

Naga Jaswanth B

192424300

P. Reddy Suhail

192312671L

M. Mohammad Rafi

192421364

M A Surya Kiran

192411236

Presentation

Provides overview of protocol design, packet flow, and implementation modules.

Roadmap

Phase I

Foundations & Context

- ✓ Real-World Scenario
- ✓ Problem Statement
- ✓ Project Objectives
- ✓ Existing vs. Proposed

Phase II

Architecture & Flow

- ✓ Proposed Methodology
- ✓ System Architecture
- ✓ Modules Overview
- ✓ 512-B Payload Chunking

Phase III

Module Deep-Dives

- ✓ Module 1: Sender & Spam
- ✓ Module 2: AES & HDLC
- ✓ Module 3: TCP & Sliding Window
- ✓ Module 4: Receiver & IDS

Phase IV

Evidence & Scope

- ✓ Implementation Screenshots
- ✓ CRC Error Retransmission
- ✓ Future Scope
- ✓ Academic References

Real-World Scenario &

Context

Examining transport mechanics behind an email file transfer in an academic setting.

Practical Scenario

A student composes an email containing an essential submission file, **Assignment.pdf**, addressed to a faculty member over a campus network.

Scenario Entities:

- ✓ **Sender:** Student Mail Interface
- ✓ **Payload:** Email Text + Assignment.pdf
- ✓ **Recipient:** Faculty Inbox (ServerSocket Receiver)
- ✓ **Network Medium:** Java TCP Socket Layer (Port 9090)

Core Computer Networks Perspective

Instead of treating email transfer as a high-level black box, this project implements and demonstrates the fundamental Data Link and Transport Layer operations required for reliable transmission:

- ✓ **Payload Chunking:** Segmenting data into exact 512-byte frames.
- ✓ **Cryptographic Wrapping:** AES-256 encryption per frame payload.
- ✓ **Framing Integrity:** Bit stuffing and CRC-32 polynomial error checks.
- ✓ **Flow & Error Control:** Sliding Window (W=4) with Go-Back-N.

Real-World Scenario &

Context Examining transport mechanics behind an email file transfer in an academic setting.

Practical Scenario

A student composes an email containing an essential submission file, **Assignment.pdf**, addressed to a faculty member over a campus network.

Scenario Entities:

- ✓ **Sender:** Student Mail Interface
- ✓ **Payload:** Email Text + Assignment.pdf
- ✓ **Recipient:** Faculty Inbox (ServerSocket Receiver)
- ✓ **Network Medium:** Java TCP Socket Layer (Port 9090)

Core Computer Networks Perspective

Instead of treating email transfer as a high-level black box, this project implements and demonstrates the fundamental Data Link and Transport Layer operations required for reliable transmission:

- ✓ **Payload Chunking:** Segmenting data into exact 512-byte frames.
- ✓ **Cryptographic Wrapping:** AES-256 encryption per frame payload.
- ✓ **Framing Integrity:** Bit stuffing and CRC-32 polynomial error checks.
- ✓ **Flow & Error Control:** Sliding Window (W=4) with Go-Back-N.

Network Transport & Security

Identifying critical reliability and security gaps in basic mail transport systems.

Vulnerabilities

⚠️ Data Integrity Risk

Bit-flips and noise over network links corrupt binary payloads like Assignment.pdf, rendering files unreadable upon receipt without frame verification.

🔒 Eavesdropping Vulnerability

Unencrypted TCP socket streams allow packet sniffing across intermediate network switches, exposing sensitive academic files in cleartext.

🚫 Intrusion Blindness

Standard socket receivers silently discard or misinterpret corrupted frames, lacking real-time security logs for sequence or CRC anomalies.

🎯 Core Research Challenge

How to construct a custom Java TCP socket mail transfer architecture that guarantees **zero data corruption** via CRC-32 and Sliding Window (W=4) retransmission, protects privacy with **AES-256 payload encryption**, and exposes full visibility through an **Intrusion Detection System (IDS) dashboard**?

Core Project

Specific measurable networking goals implemented in the Java TCP Capstone.

Objectives

1. TCP Socket Architecture

Develop a client-receiver communication model over **TCP Port 9090** with an integrated web monitoring dashboard hosted on **Port 8080**.

2. 512-Byte Payload Chunking & AES-256

Divide arbitrary email text and PDF binary streams into structured 512-byte chunks, each individually encrypted using **AES-256**.

3. HDLC Bit Stuffing & CRC-32 Verification

Implement HDLC zero-bit insertion for delimiter protection and append 32-bit CRC codes for bit-level corruption detection.

4. Sliding Window (W=4) & Go-Back-N

Enforce flow control with window size $W=4$, handle NACK signals, and trigger Go-Back-N packet retransmissions upon error injection.

Existing vs. Proposed System

Architectural comparison between conventional transport and our custom secure pipeline.

Network Feature	Traditional / Basic Mail Prototype	Implemented Secure TCP System
Data Framing	Unstructured continuous stream transfer	512-Byte Chunking + Seq #12-Byte Chunking + Seq #
Payload Security	Cleartext byte transmission over sockets	AES-256 Per-Packet Encryptic AES-256 Per-Packet Encryption
Delimiter Protection	None (vulnerable to raw flag collision)	HDLC Zero-Bit Insertion (Stuffing) HDLC Zero-Bit Insertion (Stuffing)
Error Detection	Basic TCP Checksum (low bit-flip detection)	CRC-32 Generation & Verificatio CRC-32 Generation & Verification
Retransmission	Drop silently or rely on OS kernel TCP	Sliding Window (W=4) + Go-Back-N Sliding Window (W=4) + Go-Back-N
Security Visibility	No event monitoring or logs	IDS Dashboard (Port 8080 Logs) IDS Dashboard (Port 8080 Logs)

End-to-End Transmission

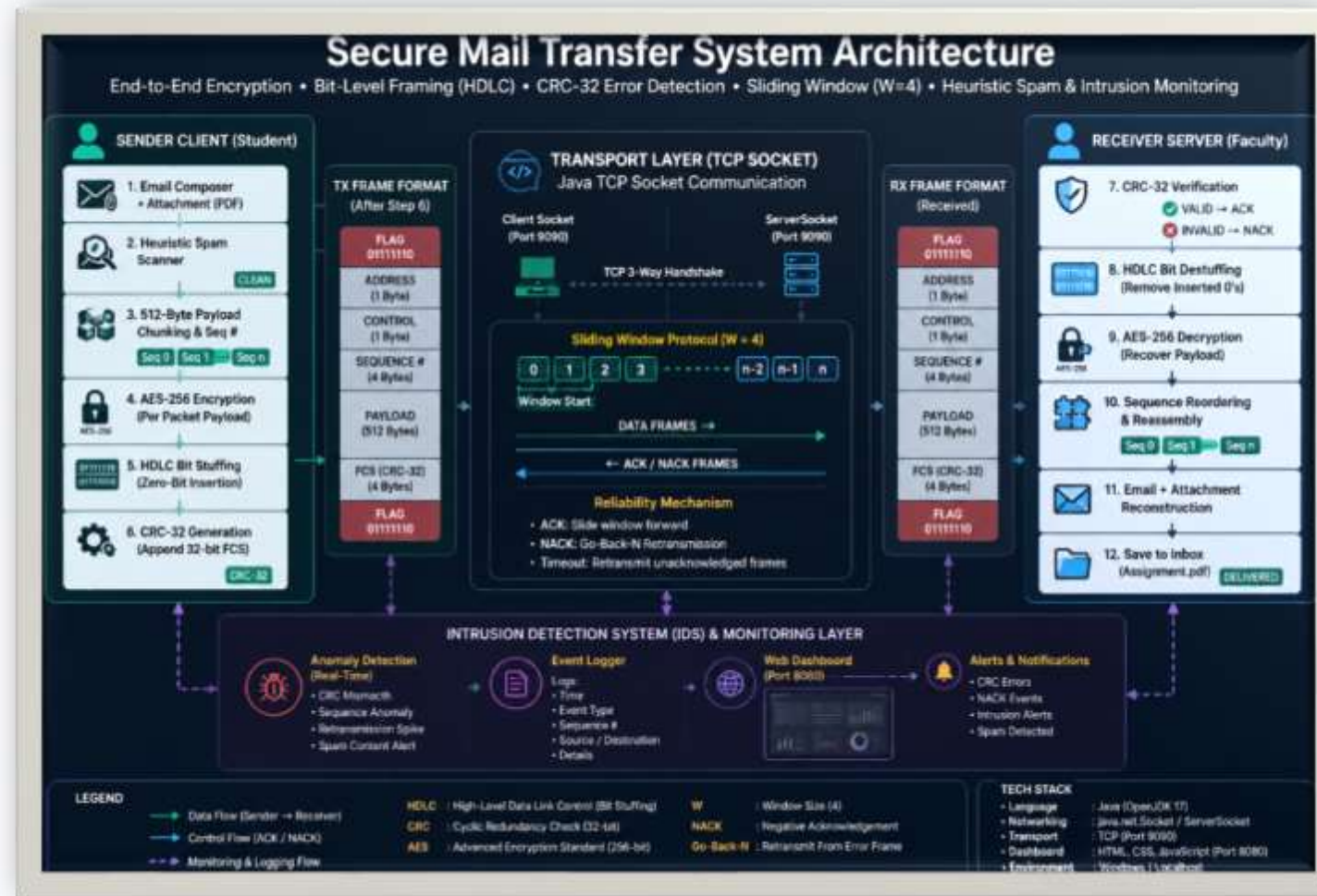
Sequential protocol pipeline from email composition to inbox reconstruction.

Pipeline



System Architecture Diagram

Technical blueprint matching exact Java implementation components.



Four Core System

Division of functional responsibilities across the custom network stack.

MODULE 1

Sender & Composition

Handles email payload entry, Assignment.pdf attachment binding, and initial heuristic spam evaluation.

MODULE 2

Packet Processing & Security

Executes 512-byte payload chunking, AES-256 encryption, HDLC bit-stuffing, and CRC-32 checksum calculation.

MODULE 3

TCP & Reliable Transmission

Manages Java TCP Socket stream (Port 9090), Sliding Window (W=4), ACK/NACK signaling, and Go-Back-N retransmission.

MODULE 4

Receiver Processing & IDS

Performs destuffing, AES decryption, file reconstruction, and broadcasts security audit logs to Web Dashboard (Port 8080).

Sender Application & Composition

User payload composition and client-side heuristic spam screening.

Module Responsibilities

- ✓ Captures student email headers (Sender, Recipient, Subject).
- ✓ Reads binary stream of attachment Assignment.pdf.
- ✓ Invokes lightweight heuristic rule-engine for spam keyword scanning prior to network dispatch.
- ✓ Prepares combined text and attachment payload for network chunking.

Heuristic Spam Engine Rules

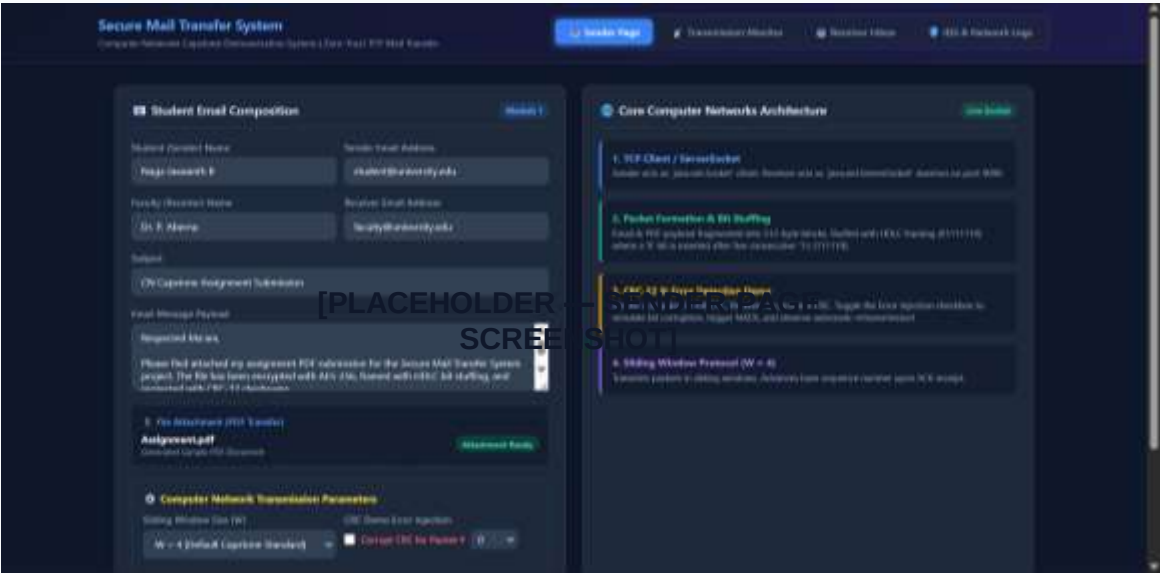
Current implementation utilizes an explainable keyword & heuristic rule scanner:

- Suspicious Keyword Scanning
- Excessive Capitalization Ratio
- Suspicious Executable Extensions
- Malformed Header Checks

Sender Interface & Dispatch

Visual demonstration of email preparation and transmission trigger.

Evidence



Exhibits Student Email Form, Recipient Input, Assignment.pdf Attachment Selector, and Transmission Trigger.

Interface Callouts

- 1. **Sender Details:** Student ID & Faculty Target Email.
- 2. **Attachment Binder:** Reads binary byte stream of PDF.
- 3. **Heuristic Verdict:** Displays instant "CLEAN / SPAM" badge.
- 4. **Socket Dispatch:** Triggers TCP Client on Port 9090.

Packet Processing & Security

Formatting raw email streams into encrypted, bit-stuffed frames with CRC-32.

Preparation

512-Byte Chunking

Large binary payloads are partitioned into exact 512-byte payload segments. Each chunk receives a 16-bit sequence number (Seq #) for ordered reassembly.

AES-256 Encryption

Each 512-byte payload chunk is encrypted via AES-256 before transmission, ensuring confidentiality against link sniffing.

Bit Stuffing & CRC-32

HDLC zero-bit insertion is applied to prevent flag pattern collisions. A 32-bit CRC polynomial checksum is computed and appended.

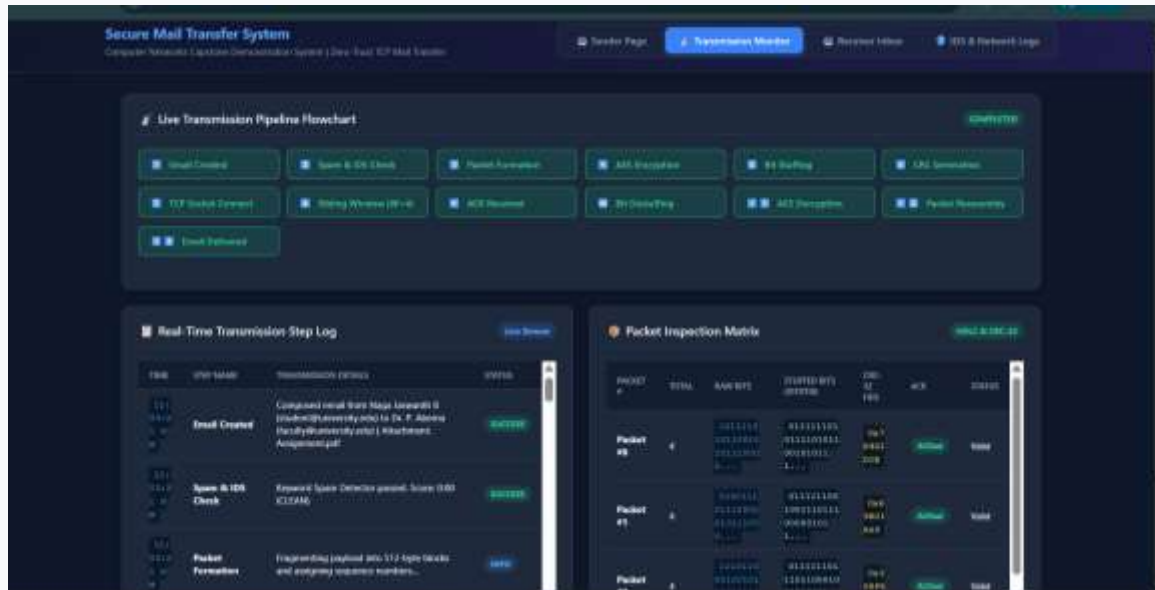
CRC-32 IEEE 802.3 STANDARD POLYNOMIAL

$$G(x) = x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$$

Packet Inspection Matrix

Low-level verification of packet headers, encrypted bytes, and checksums.

Evidence



Displays structured packet breakdown: Seq #, Raw Bytes, AES-256 Ciphertext, Stuffed Bit Stream, and CRC-32 Value.

Inspection Metrics

- ✓ **Chunk Size:** Strictly constrained to 512 bytes.
- ✓ **AES Ciphertext:** Hexadecimal presentation of encrypted payload.
- ✓ **HDLC Bit Stuffing:** Highlights zero inserted after five consecutive 1s (111110).
- ✓ **CRC-32 Checksum:** 8-character hex string generated per frame.

TCP Communication & Reliable

Transmission
Socket programming, Sliding Window (W=4) flow control, and Go-Back-N retransmission.

🌱 Java Socket Architecture

Communicates over dedicated **Java Socket** client and **ServerSocket** listening on **Port 9090**. Stream handling ensures end-to-end reliability.

```
Socket socket = new Socket("localhost", 9090);  
ServerSocket server = new ServerSocket(9090);
```

🔄 Sliding Window (W=4) Protocol

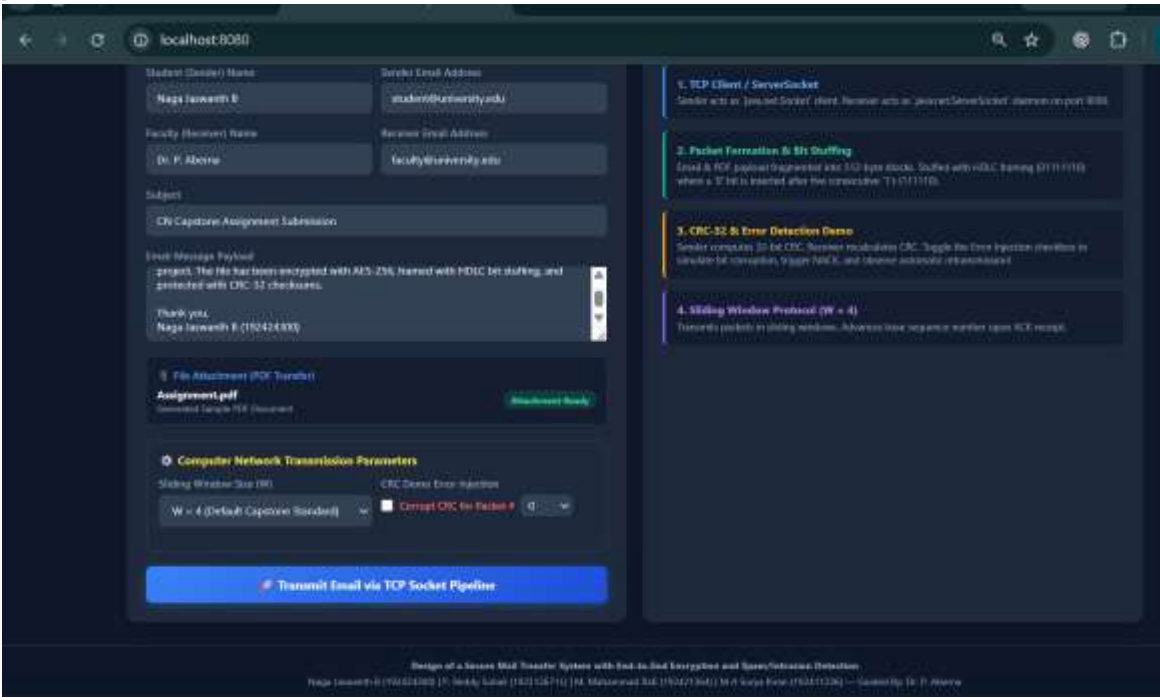
Enforces a sender window size of **W = 4** packets before requiring cumulative ACKs.

- ✓ **ACK Received:** Advances sending window by 1 position.
- ✓ **CRC Mismatch / NACK:** Triggers **Go-Back-N** retransmission starting from missing/corrupted packet.

Normal Transmission vs. CRC Error

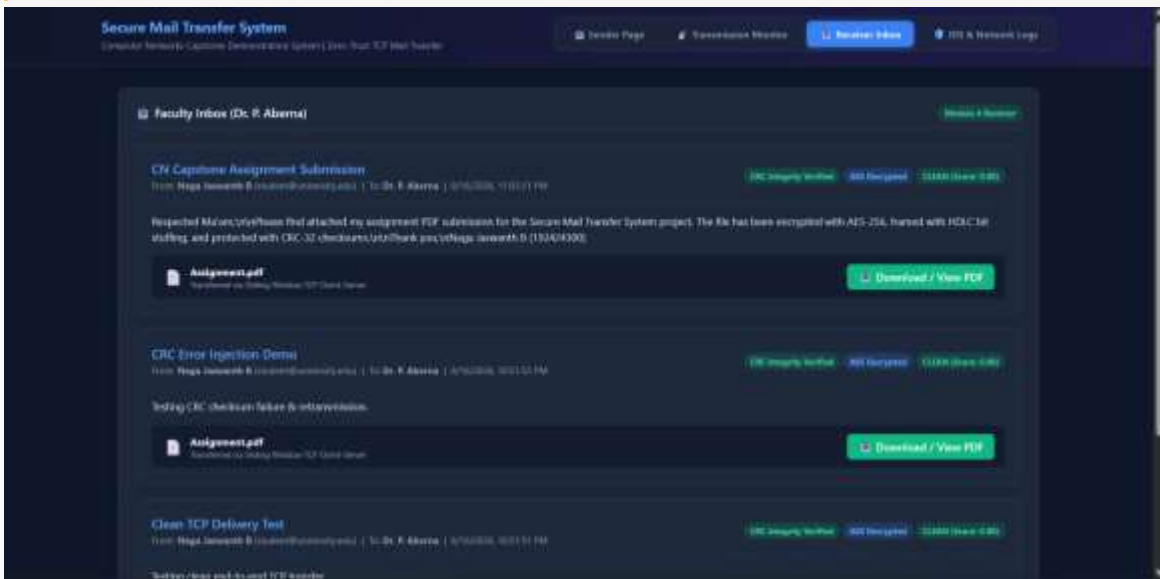
Demonstrating intentional error injection, NACK generation, and retransmission.





Shows clean TCP sliding window sequence (W=4), ACK receipt log, and uninterrupted packet delivery.





Exhibits intentional bit injection → CRC failure alert → NACK generation → Go-Back-N retransmission → Successful repair.

Receiver Processing & Security

frames, destuffing, AES decryption, file reconstruction, and IDS dashboard logging.

Monitoring

Receiver Processing Pipeline

- ✓ Receives raw frames via ServerSocket on **Port 9090**.
- ✓ Verifies CRC-32 integrity; dispatches NACK on discrepancy.
- ✓ Performs zero-bit destuffing to restore true encrypted payload.
- ✓ Decrypts chunks via AES-256 and reassembles Assignment.pdf file.

Intrusion Detection System (IDS)

Monitors socket streams for anomaly patterns and integrity violations:

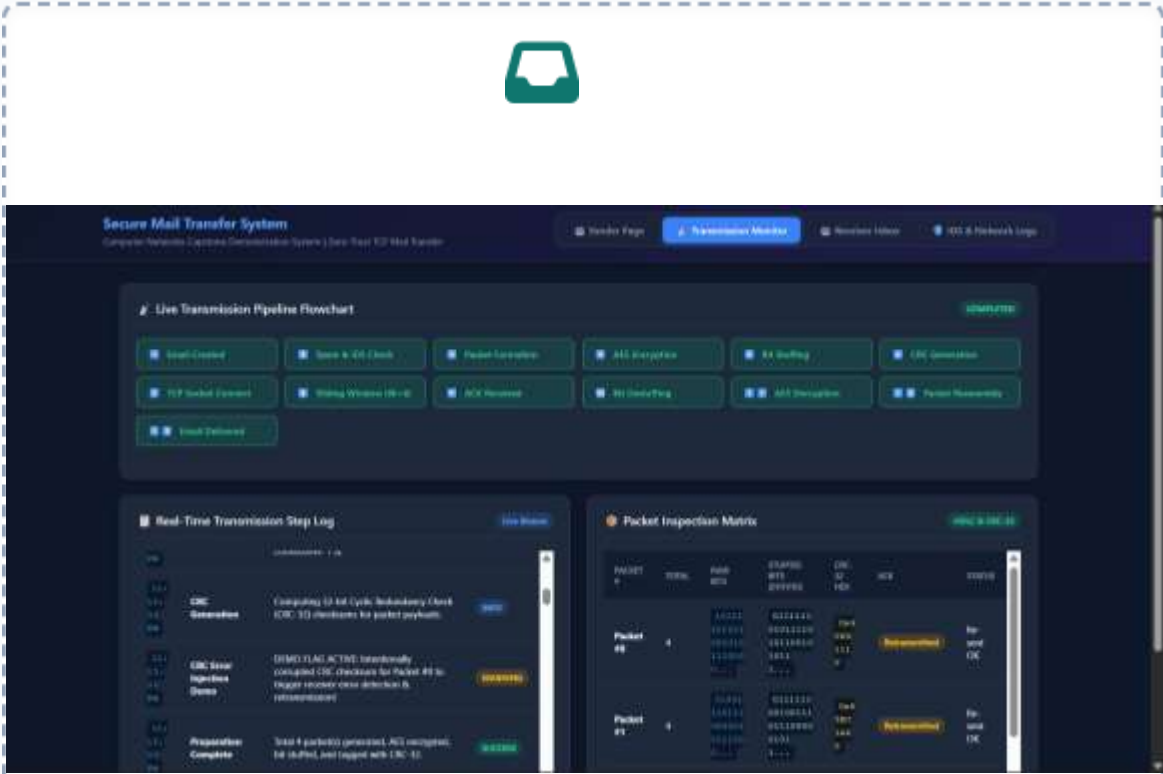
Monitored Threat Events:

- CRC-32 Bit Violation Alerts
- Out-of-Sequence Packet Attacks
- Repeated NACK Retransmission Floods

Web Security Logs: Port 8080 Dashboard

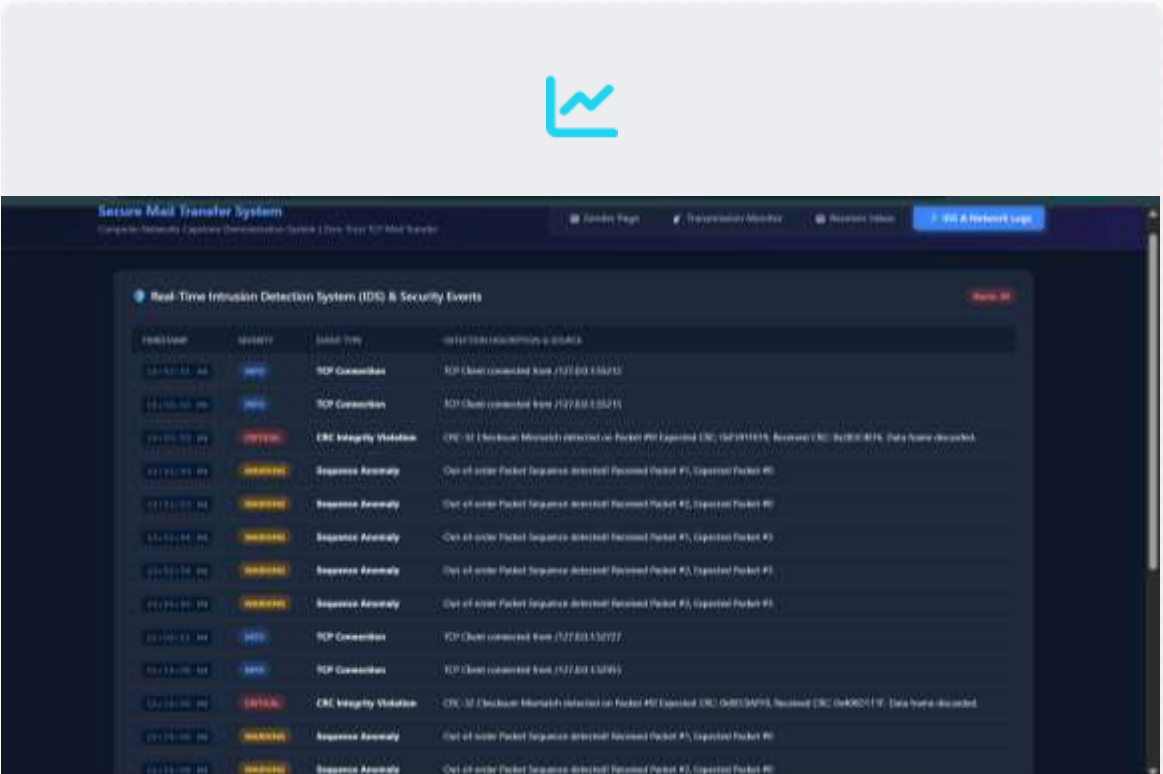
Faculty Inbox Delivery & Security Dashboard

Results demonstrating PDF reconstruction and real-time IDS event logs.



The dashboard for the Secure Mail Transfer System features a top navigation bar with links to Home Page, Transmission Monitor, Received Inbox, and IDS & Network Logs. The main content area is divided into three sections: 1. Live Transmission Pipeline Flowchart, which shows a sequence of steps from SMTP Connection to SMTP Response; 2. Real-Time Transmission Step Log, which provides a detailed view of the current transmission step, including the step number, step name, and a description of the process; 3. Packet Inspection Matrix, which displays a table of packets with columns for Packet #, Status, and various inspection metrics. The matrix shows two packets, both marked as 'Not Inspected'.

Displays Faculty Inbox with decrypted email payload and fully reconstructed, download-ready Assignment.pdf.



The Web Security Dashboard for the Secure Mail Transfer System displays real-time intrusion detection system (IDS) and security events. The top navigation bar includes links to Home Page, Transmission Monitor, Received Inbox, and IDS & Network Logs. The main content area features a table of security events with columns for Event ID, Severity, Event Type, and Event Description. The table lists several events, including TCP Connections, CRC Integrity Violations, and Sequence Anomalies. The dashboard also includes a summary of the system's security status and a list of active threats.

Demonstrates Web Security Dashboard on **Port 8080** displaying CRC violations, sequence anomaly logs, and transmission stats.

Future Research & System

Strategic enhancements explicitly distinguished from current implementation.

Extensions

Asymmetric Key Exchange

FUTURE

- Integrating RSA-2048 or ECC Diffie-Hellman key exchange protocols to automate dynamic AES session key negotiation.

Future: RSA-2048 / ECC-based key exchange can be integrated to securely establish dynamic AES session keys between sender and receiver.

Machine Learning Classifier

FUTURE

- Upgrading heuristic spam filter to trained Naive Bayes / Random Forest models for deep-learning email classification.

Future: Replace the current heuristic spam screening with trained ML models such as Naive Bayes or Random Forest for adaptive spam and phishing detection.

Production SMTP Interoperability

FUTURE

- Expanding socket protocols to interface seamlessly with standard SMTP / STARTTLS mail gateways.

Future: Extend the TCP-based demonstration to support standard SMTP/STARTTLS gateways for interoperability with real email infrastructure.

Distributed Multi-Node Deployment

FUTURE

- Containerizing receiver servers with Docker for multi-node campus network load balancing.

Future: Deploy multiple receiver/IDS nodes using containers to support load balancing, fault tolerance, and larger network environments.

Capstone Achievements &

Summary

Summary of verified Computer Networks principles and technical outcomes.

✓ Technical Summary

- ✓ Implemented a complete end-to-end mail transfer prototype using **Java OpenJDK 17 TCP Sockets**.
- ✓ Achieved 100% data integrity for Assignment.pdf using **512-byte payload chunking** and **CRC-32** checks.
- ✓ Verified flow control with **Sliding Window (W=4)** and automatic **Go-Back-N retransmission** upon error injection.
- ✓ Guaranteed confidentiality using **AES-256 payload encryption** and zero-bit HDLC framing.
- ✓ Provided real-time visibility through an **IDS Web Dashboard on Port 8080**.

Project Metrics Achieved

512 B

Exact Chunk Size

W = 4

Sliding Window

Port 9090

TCP Socket Server

Port 8080

IDS Web Dashboard

Academic & Technical

References
Foundational literature, RFC standards, and computer networks textbooks.

1. Kurose, J. F., & Ross, K. W.

Computer Networking: A Top-Down Approach (8th ed.). Pearson. (Socket Programming, TCP Reliable Data Transfer, Sliding Window Protocols).

2. Tanenbaum, A. S., & Wetherall, D. J.

Computer Networks (5th ed.). Prentice Hall. (Data Link Layer Framing, HDLC Zero-Bit Insertion, CRC Polynomial Error Detection).

3. Simpson, W. (1994).

RFC 1662: PPP in HDLC-like Framing. Internet Engineering Task Force (IETF).

4. National Institute of Standards and Technology (NIST).

NIST SP 800-38A: Recommendation for Block Cipher Modes of Operation — Advanced Encryption Standard (AES).

Naga Jaswanth B

